

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Dave's Taxonomy: Articulation (Level 4)

Total Marks: 25

Question Count: 5

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

NAME:K.Saipriya

REGNO: 192421296

COURSECODE: CSA1510

COURSENAME:CloudComputing

1. Hadoop Job Fails Because Input Splits Are Uneven and Reducers Remain Idle

Error Identification

The likely design error is poor input partitioning and data distribution. The input files are probably divided into highly uneven splits, causing some Mapper tasks to receive much more data than others.

This creates a data-skew problem.

For example:

Input Data

```
|
+---- Split 1 → 10 MB → Mapper 1 → finishes quickly
|
+---- Split 2 → 10 MB → Mapper 2 → finishes quickly
|

+---- Split 3 → 500 MB → Mapper 3 → takes much longer
```

The other tasks finish early and remain idle while Mapper 3 continues processing.

Reducers may also remain idle if the Map phase produces insufficient or poorly distributed intermediate data.

Root Cause

The main causes may include:

- Uneven input file sizes.
- Poor input splitting.
- Data skew.
- Incorrect partitioning.
- A small number of large files.
- Poor reducer configuration.
- Unbalanced intermediate keys.

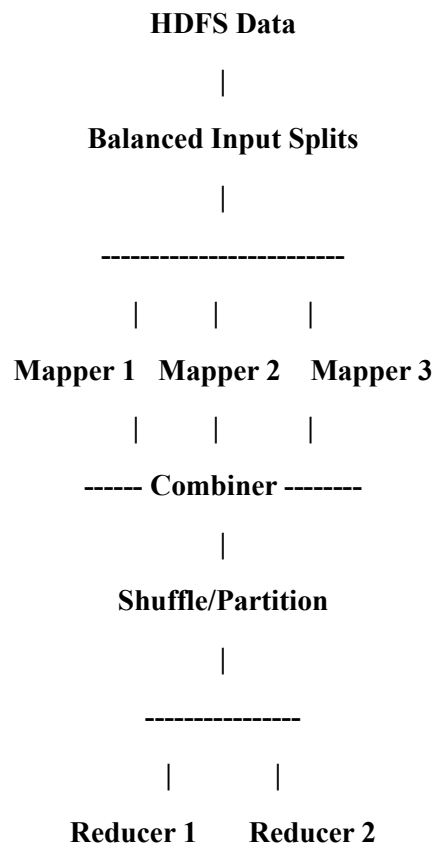
Hadoop's performance depends on distributing work across the available nodes. If one task receives significantly more data, it becomes a **straggler task**.

Corrective Actions

1. Use appropriate HDFS block sizes.
2. Split very large input files into balanced chunks.
3. Use a suitable InputFormat.
4. Improve key partitioning so data is distributed evenly.
5. Use a **Combiner** where applicable to reduce intermediate data.
6. Configure an appropriate number of Reducers.
7. Monitor task execution time to identify stragglers.
8. Use speculative execution when slow tasks are caused by abnormal node performance.

Hadoop Concept Application

The corrected workflow becomes:



Justification

Balanced input splits ensure that Mapper tasks receive approximately equal workloads. Proper partitioning distributes intermediate keys across Reducers, preventing one Reducer from becoming a bottleneck.

Therefore, correcting the input and partitioning strategy improves parallelism and prevents Reducers from remaining unnecessarily idle.

2. HDFS Data Becomes Unavailable After a Single Node Failure

Error Identification

The most likely design error is insufficient HDFS replication or poor NameNode availability design.

If one DataNode fails and the data becomes unavailable, the system may have:

- Replication factor of 1.
- Replicas placed incorrectly.
- Insufficient healthy DataNodes.
- A single point of failure at the NameNode.

Root Cause

HDFS is designed to tolerate DataNode failures by maintaining multiple copies of data blocks.

For example, with replication factor 3:

Block A

|— DataNode 1

|— DataNode 2

|— DataNode 3

If DataNode 1 fails:

DataNode 1 → FAILED

DataNode 2 → Available

DataNode 3 → Available

The data should still be accessible.

If there is only one copy:

Block A → DataNode 1 → FAILED

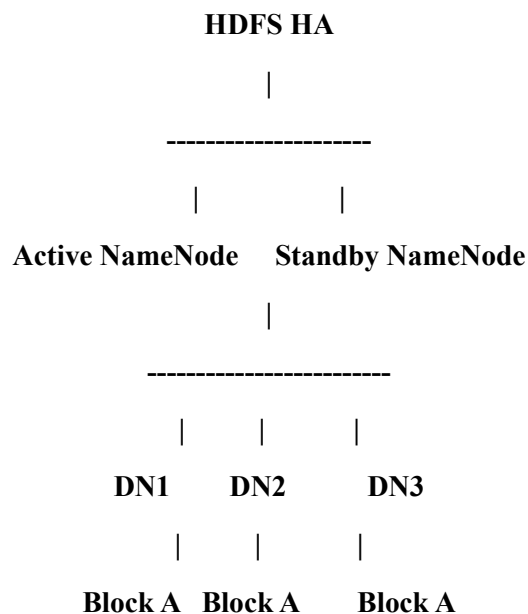
the data becomes unavailable.

Another possible problem is a **single NameNode**. Even when DataNodes are healthy, NameNode failure can make the HDFS namespace unavailable.

Corrective Actions

- Set an appropriate HDFS replication factor, typically 3 for critical data.
- Use rack-aware replica placement.
- Deploy Active and Standby NameNodes.
- Configure automatic failover.
- Monitor DataNode health.
- Allow HDFS to automatically re-replicate blocks after failures.
- Maintain sufficient DataNodes to hold replicas independently.

Correct Architecture



Justification

Replication prevents the failure of one Data Node from causing data loss or unavailability. NameNode High Availability prevents the Name Node from becoming a single point of failure.

Thus, Data Node replication + rack awareness + Name Node HA provides much stronger fault tolerance.

3. ETL + Apache NiFi Creates Duplicate Records

Error Identification

The root problem is likely a lack of duplicate detection and idempotent data ingestion.

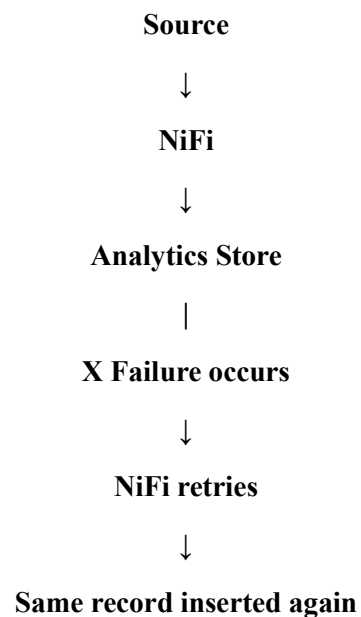
The same source record may be processed multiple times by NiFi or the ETL pipeline and inserted again into the analytics store.

Possible Causes

Common causes include:

- NiFi retries after a temporary failure.
- The same file is processed more than once.
- ETL jobs perform full loads instead of incremental loads.
- No unique key is defined.
- No record-level deduplication is performed.
- Failure occurs after data insertion but before the pipeline records successful completion.
- Multiple ingestion pipelines process the same source data.

For example:



Corrective Actions

1. Use Unique IDs

Each record should have a unique business key such as:

Customer_ID + Transaction_ID

The analytics database should enforce uniqueness.

2. Use Incremental Loading

Instead of loading the complete dataset repeatedly:

Full Load

use:

New/Changed Records → Analytics Store

using timestamps, transaction IDs, or Change Data Capture.

3. Use NiFi Deduplication

NiFi can track identifiers or use record-processing mechanisms to prevent previously processed records from being inserted again.

4. Use Upsert

Instead of always performing:

INSERT

use:

INSERT OR UPDATE

when the data model requires it.

5. Track Processing State

Maintain metadata such as:

File ID

Record ID

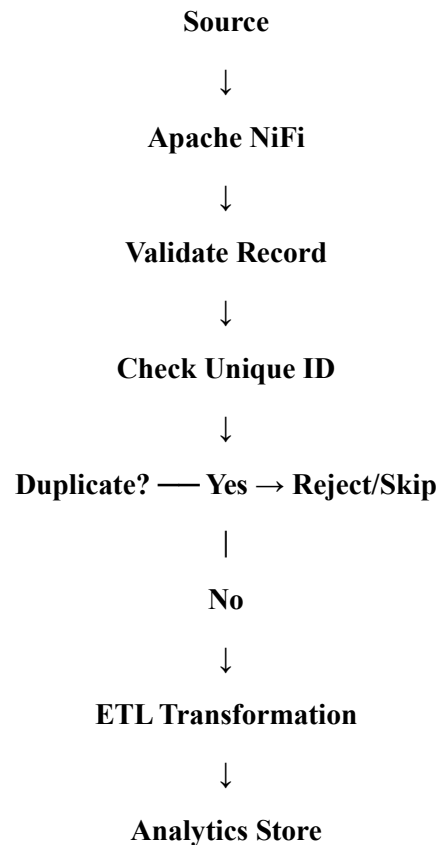
Source

Timestamp

Processing Status

Checksum

Correct Workflow



Justification

Deduplication at both the pipeline and storage layers makes the system more reliable. Unique constraints protect the analytics store even if the ingestion pipeline accidentally sends the same record twice.

Therefore, the main solution is idempotent ingestion + unique keys + incremental loading + duplicate detection.

4. YARN Resource Starvation When Multiple Users Submit Jobs

Error Identification

The scheduling error is likely that the YARN cluster does not have an effective queue-based resource management and fairness policy.

One user or workload may consume most of the cluster resources, leaving insufficient CPU and memory for other users.

Root Cause

A poorly configured scheduler may:

- Allow unlimited resource consumption.
- Lack user limits.
- Have no queue structure.
- Assign excessive resources to a single job.
- Lack minimum resource guarantees.
- Not prioritize workloads appropriately.

Example:

Total Cluster Resources = 100%

User A → 80%

User B → 15%

User C → 5%

User A can starve other users.

Corrective Action

Use the YARN Capacity Scheduler or Fair Scheduler-style queue management, depending on the cluster configuration.

For example:

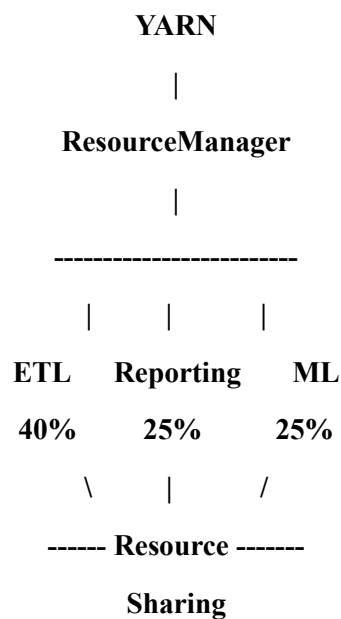
Queue	Resource Capacity	Purpose
ETL	40%	Data processing
Reporting	25%	Business reports
ML	25%	Machine learning
General	10%	Other users

Set:

- Queue capacity.
- Maximum queue capacity.

- Per-user limits.
- Application priorities.
- Minimum resource guarantees.

Improved Architecture

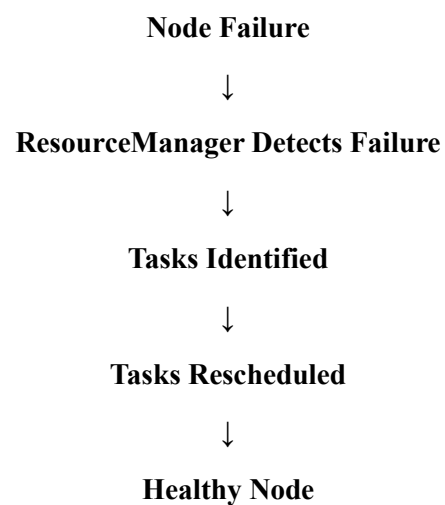


Unused resources can be temporarily allocated to other queues without allowing one user to permanently monopolize the cluster.

Failure Handling

The ResourceManager monitors NodeManagers.

If a node fails:



Justification

Queue-based resource management prevents one user from consuming the entire cluster. Capacity guarantees provide fairness, while priority controls allow important workloads to receive resources when required.

Thus, the solution addresses resource starvation, fairness, priority, and efficient cluster utilization.

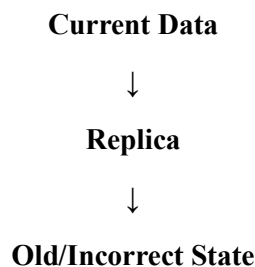
5. Backup Restores Outdated Data After Recovery

Error Identification

The probable design flaw is that the backup system is using stale backups or an incorrect replication strategy.

Replication alone should not be treated as a complete backup solution.

For example:



If the backup process has not captured recent changes, recovery may restore an older version.

Root Cause

Possible causes include:

- Backups are performed too infrequently.
- Backup jobs fail silently.
- Only one backup version is maintained.
- Replication is asynchronous and recovery uses a stale replica.
- No backup verification is performed.
- No timestamps/version information is maintained.
- A corrupted or deleted file is replicated before the backup system captures a safe version.

Corrective Actions

1. Use Versioned Backups

Maintain multiple backup generations:

Backup 1 → Monday

Backup 2 → Tuesday

Backup 3 → Wednesday

Backup 4 → Thursday

This allows recovery from an appropriate point in time.

2. Use Incremental Backups

Perform:

Full Backup + Incremental Changes

rather than relying only on occasional full backups.

3. Verify Backups

After creating a backup, automatically verify:

- File existence.
- Checksums.
- Timestamp.
- Backup completeness.
- Restore capability.

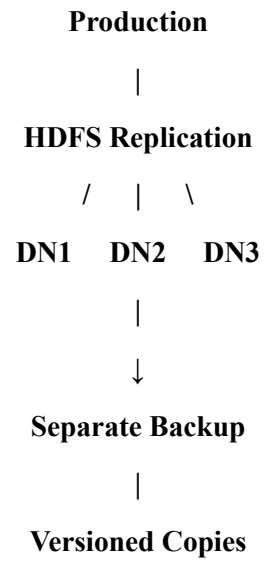
4. Maintain Recovery Points

Define appropriate Recovery Point Objectives (**RPO**) so that backups are frequent enough to meet business requirements.

5. Separate Replication from Backup

HDFS replication protects against node failure, but it does not replace historical backups.

For example:



If a file is accidentally deleted and the deletion is replicated to all three Data Nodes, replication cannot recover the deleted historical version.

Justification

A proper backup architecture should maintain multiple historical recovery points, not just current replicas. Backup verification ensures that stored copies can actually be restored.

Therefore, separating HDFS replication for availability from versioned backup for recovery prevents outdated or unusable recovery data.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented